

# CSCI 112

## Programming with C

### LECTURE: 11

J. L. Pach

# OUTLINE:

- Pointers
  - Review
  - Summary
- Function arguments
  - by the value
  - by the pointer
- `const` keyword
- Pointer operations

# LECTURE 02

slide 27

# POINTERS ARE TREATED AS FIRST-CLASS DATA TYPES

```c
int main()
{
  short int p;        /* Declaration of variable p with a size of 2 bytes */
  short int * q = &p; /* Declaration and initialization of pointer q with a size of 4 bytes
                      (even though it points to short int) */
  float*r;            /* Declaration of pointer r to float  */
  char* s;            /* Declaration of pointer s to char  */
  int t, *v;          /* Declaration of variable t and pointer v */
  short int* w, z;    /* Declaration of pointer w to short int and variable z */

/* "Note that the variable type is determined by the position of the asterisk ('*') in the declaration.
Only the variable directly following the asterisk is considered a pointer. */
}
```

We can create a pointer to **any** data type using the `*` operator between the existing data type and the symbolic name. Unary Operator `&` returns memory locations. A pointer is a reference to a specific memory location

# LECTURE 04

slide 19

# SUMMARY OF MEMORY SIZE OF DATA TYPES

| Type | Memory size in bytes / bits |
|---|---|
| `char` | 01 Bytes / 08 bits |
| `bool` * | 01 Bytes / 08 bits |
| `short int` | 02 Bytes / 16 bits |
| `long int` | 04 Bytes / 32 bits |
| `float` | 04 Bytes / 32 bits |
| `double` | 08 Bytes / 64 bits |
| `long double` | 12 Bytes / 96 bits |
| pointer ( `char` *, `short` *, `long` *, `int` *, `float` *, `double` *, `long double` *, `void` *, …) | 04 Bytes / 32 bits |
| user defined ( `struct` , `union` , etc. ) | complex |

*from C99

# LECTURE 05

slide 19

# MAGIC...

```c
int main()
{
/* 01 */            int x = 10;
/* 02 */            int * label_pointer;
/* 03 */    label_pointer = ( x > 0 ) ? &&label_greater: &&label_lower; /* equals 08 or 12 */
/* 04 */            goto *label_pointer;
/* 05 */    label_greater:
/* 06 */            printf("%s\n","a variable x is greater than 0");
/* 07 */            goto end_label;
/* 08*/     label_lower:
/* 09 */            printf("%s\n","a variable x is lower than 0");
/* 11 */            goto end_label;
/* 12 */    end_label:
/* 13 */            ;
}
```



# LECTURE 06

slide 16



```c
#include <stdio.h>                    /*global vs local*/
int x = 5;            /* global x */

void myFunction1(void);
void myFunction1()
{
  printf("x equals %d that is read by function myFunction\n", x);
}
int main()
{
  int * pointerGlobalX = &x;
  myFunction1();
  int x = 3;          /* local x */
  printf("x equals %d that is read by function main\n", x);
  printf("x equals %d that is read by function main\n", *pointerGlobalX);
  return 0;
} /* We don't have direct access to a shadowed variable. The only way to access it is through a pointer. */
```

```
x equals 5 that is read by function myFunction
x equals 3 that is read by function main
x equals 5 that is read by function main
```

# POINTERS

## Complete Summary & Consequences

# 1. POINTERS ARE FIRST-CLASS DATA TYPES

- You can declare pointers to **any** data type
- Pointers use the `*` operator in declarations
- The `&` operator returns memory address
- **The asterisk position matters**: only the variable directly after `*` becomes a pointer

```c
int t, *v;          /* t is int, v is pointer to int */
short int* w, z;    /* w is pointer to short int, z is short int */
```



# 2. MEMORY SIZE CONSEQUENCES

| Type | Size |
|---|---|
| char | 1 byte |
| short int | 2 bytes |
| int / long int | 4 bytes |
| float | 4 bytes |
| double | 8 bytes |
| long double | 12 bytes |
| **ANY pointer type** | **4 bytes** (32-bit system) |

**Consequence**: All pointers have the SAME size, regardless of what they point to!

```c
char* p1;       /* 4 bytes */
double* p2;     /* 4 bytes */
int* p3;        /* 4 bytes */
```



# 3. OPERATORS

| Operator | Name | Effect |
|---|---|---|
| `&` | Address-of | Returns memory address of variable |
| `*` | Dereference | Accesses value at pointer's address |

```c
int x = 10;
int* p = &x;       /* p points to x (contains address of x) */
printf("%d", *p);   /* Prints 10 (dereferences p to get x's value) */
```



# 4. GLOBAL VS LOCAL SCOPE - VARIABLE SHADOWING

When a local variable has the same name as a global variable:

- **Local variable "shadows" the global variable**
- The global variable becomes inaccessible by name
- **But**: You can still access it through a pointer saved before shadowing!

```c
int x = 5;              /* Global x */

int main()
{
    int* globalXPtr = &x;   /* Save pointer to global x */
    int x = 3;              /* Local x shadows global x */
    printf("%d\n", x);              /* Prints 3 (local x) */
    printf("%d\n", *globalXPtr);    /* Prints 5 (global x via pointer) */
}
```



# PRACTICAL CONSEQUENCES 1/2

## 1. Pointer Declaration Rules

- `int *p, q;` → `p` is pointer, `q` is `int`
- `int* p, q;` → `p` is pointer, `q` is `int`
- Always place `*` next to the variable name you want to make a pointer

## 2. Memory Address Knowledge

- All pointers occupy 4 bytes (on 32-bit systems)
- The data they point to can be any size
- Two different pointers can point to different-sized objects

# PRACTICAL CONSEQUENCES 2/2

## 3. Scope Circumvention

- Pointers allow access to shadowed variables
- Pointers can escape local scope by storing addresses
- Pointers can outlive the function they were created in

## 4. Type Safety

- `char*` and `double*` are different types
- The compiler tracks what each pointer points to

> **Dereferencing a `char*` gives 1 byte, dereferencing `double*` gives 8 bytes**

# WARNING: ADVANCED USAGE

Pointers enable powerful techniques like:

- **Computed goto** (pointer to label)
- **Function pointers** (pointers to functions)
- **Complex data structures** (linked lists, trees)

But also create risks:

- **Null pointer dereference** → crash
- **Use-after-free** → undefined behavior
- **Buffer overflow** → security vulnerability

# SUMMARY TABLE: DECLARATION PATTERNS

| Declaration | What It Means |
|---|---|
| `int x;` | x is an integer |
| `int* p;` | p is a pointer to int |
| `int *p, q;` | p is pointer to int, q is int |
| `int* p, *q;` | p and q are both pointers to int |
| `char* s;` | s is a pointer to char |
| `int** pp;` | pp is a pointer to a pointer to int |

# KEY TAKEAWAYS

- **Pointers are first-class types** — you can declare, pass, return them
- **All pointers are 4 bytes** — size doesn't depend on what they point to
- `&` **finds addresses,** `*` **reads values** — they're inverses of each other
- **Shadowing is solvable with pointers** — save the address before shadowing
- **Position of** `*` **matters** — `int *p, q;` is different from `int* p, q;`
- **Pointers escape scope** — addresses can be stored and used elsewhere

# THE DEREFERENCE OPERATOR: UNARY VS BINARY 1/3

## BINARY OPERATOR (MULTIPLICATION)

```c
int a = 5;
int b = 3;
int result = a * b;       /* Binary: multiply a and b <-> Compiler sees: result = mul(a, b) */
```



## UNARY OPERATOR (DEREFERENCE)

```c
int x = 5;
int *ptr = &x;
*ptr = 10;              /* Unary: dereference ptr - Think of it as: getValue(int* ptr) */
```



> **Key Insight**: The asterisk `*` in `*ptr` is NOT multiplication!
> It's a unary operator that "follows the address" and `gets` / `sets` the value.

# THE DEREFERENCE OPERATOR: UNARY VS BINARY 2/3

## Mental Model

```
Binary operator (*):
    a * b  ===  compile-time:  mul(a, b)

Unary operator (*):
    *ptr  ===  compile-time:  getValue(ptr)

    where getValue() means:
    "follow the address stored in ptr and return the value"
```

# THE DEREFERENCE OPERATOR: UNARY VS BINARY 3/3

## Example with Visualization

```c
int x = 5;
int *ptr = &x;            /* ptr contains address 1000 */

*ptr = 10;               /* Dereference: follow address, change value */
                        /* Compiler: go to address 1000, write 10  like `setValue` */

printf("%d", *ptr);       /* Dereference: follow address, read value */
                        /* Compiler: go to address 1000, read value */
                        /* Output: 10 */

printf("%p", ptr);        /* Print the ADDRESS itself (not dereferenced) */
                        /* Output: 1000 - in hex*/
```

# NEW

material

# FUNCTION ARGUMENTS

> When you invoke a function with an argument, the compiler creates a `NEW` variable and initializes it with a **copy** of that argument's value, even though it has the same value, is a completely different variable!

```c
void byTheValue(int value)
{   //int value = x;
    //body
    return;
}
int main()
{
   byTheValue(x); // byTheValue( int value = x;)
   return 0;
}
```



```c
int main()
{
   int x, y, z;
   x = 3;   /* x = 3; y = ?; z = ? */
   y = x;   /* x = 3; y = 3; z = ? */
   z = 2;   /* x = 3; y = 3; z = 2 */
   x = 1;   /* x = 1; y = 3; z = 2 */
   return 0;
}
```



# BY THE VALUE - EXAMPLE

```c
#include <stdio.h>
void byTheValue(int);
void byTheValue(int value)
{
  value++;
  return;
}
int main()
{
  int x = 5;
  printf("%d\n", x);
  byTheValue(x);
  printf("%d\n", x);
  return 0;
}
```



```
Result:
5
5
```

# BY THE REFERENCE

```c
#include <stdio.h>
void byTheReference(int*);
void byTheReference(int* pointer)
{
    (*pointer)++; // NO: *pointer++;
    return;
}
```



```c
int main()
{
    int x = 5;
    printf("%d\n", x);
    byTheReference(&x);
    printf("%d\n", x);
  return 0;
}
```



```
Result:
5
6
```

> Function arguments are always **copies** of our variables, and **not** the same memory areas, a function argument, even though it has the same value, is a completely different variable!

# RELATIONS BETWEEN POINTERS AND ARRAYS

## So, what does it mean?

```c
int main()
{
  int x = 5, y = 7, z = 9;
  int * pointer = &y;
  printf("%-4s equals %d.\n", "x", &x);                 // Result: "x    equals 6422292."
  printf("%-4s equals %d.\n", "y", &y);                 // Result: "y    equals 6422288."
  printf("%-4s equals %d.\n", "z", &z);                 // Result: "z    equals 6422284."
  printf("%-4s equals %d.\n", "pointer", pointer);      // Result: "pointer equals 6422288."
  printf("%-4s equals %d.\n", "pointer*", *pointer);    // Result: "pointer* equals 7."
  printf("%-4s equals %d.\n", "pointer[1]", pointer[1]); // Result: "pointer[1] equals 5.
  return 0;
}
```

# SO, WHAT DOES IT MEAN?

```
int a[10];

a[i] == *(a+i)      <=== >      &a[i] == &*(a+i) == a+i
```

- When performing arithmetic operations on pointers, the array index is automatically multiplied by the size of the data type pointed to by the pointer.
- For example, if the array stores char values, the index is multiplied by 1, and if it stores int values, the index is multiplied by 4 (assuming an int is 4 bytes).

# CONCLUSIONS

- The expression `a[i]` is transformed by the compiler into the form `*(a+i)`.
- The square brackets following the `symbolic_name` do not provide any information about whether we are referring to an array.
- When performing arithmetic operations on pointers, the array index is automatically multiplied by the size of the data type pointed to by the pointer.

# CONST KEYWORD

- We can use const before declaration (and initialization):
  - **Array**: Prevents changing the content of the array.
  - **Pointer**:
    - Before the asterisk(*): Prevents changing the value it points to.
    - After the asterisk(*): Prevents changing the pointer itself.
- The compiler ensures the immutability of constants. It will not compile the file if you try to change their value. **However, attempting to bypass the restriction (e.g., using a pointer) will always result in incorrect program behavior without informing the programmer.**

# CONST KEYWORD IN POINTER

```c
int main()
{
  int const val = 0;
  val = 1; /* WRONG! */
  int x = 5, y = 7, z = 9;
  int const * const pointerX = &x; /* Both the pointer and the pointed-to value are constant */
  /* pointerX = &y; WRONG! */
  /* *pointerX = 1; WRONG! */
  int const *  pointerY = &y; /* The pointed-to value is constant, but the pointer can be reassigned */
  /* *pointerY = 1; WRONG! */
  pointerY = &z;
  int  * const pointerZ = &z; /* The pointer is constant, but the pointed-to value can be changed */
  *pointerZ = 1;
  /* pointerZ = &x; WRONG! */
  printf("x = %d, y = %d, z = %d\n", x, y, z ); // Result: x = 5, y = 7, z = 1
  return 0;
}
```

# CONST KEYWORD IN ARRAY

```c
#include <stdio.h>
int main()
{
  int const a[] = {1, 2, 3, 4, 5, 6, 7};
  /* a[1]=a[0]; WRONG! */
  return 0;
}
```



- The most similar thing to the `symbolic_name` of an array in C is a `pointer with the const` keyword placed **after** the asterisk, as this prevents changes to what the pointer points to.
- However, it is important to remember that there is still a small difference. Specifically, the `&` operator on a static pointer will return the address of the pointer itself, whereas for the `symbolic_name` of an array, it will return the address of the first element.

# CONCLUSIONS

```c
#include <stdio.h>
void byTheReference(int arr[], int n);
void byTheReference(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
int main()
{
    int a[] = {1, 2, 3, 4, 5, 6, 7};
    byTheReference(a, 7); // Result: 1 2 3 4 5 6 7
    getchar();
    return 0; //return r
}
```



There are no restrictions on passing an array as a function parameter. However, there's a common misconception: when we write `int arr[]`, it may seem like the entire array is copied and we get access to an identical copy. This is not true. The square bracket syntax is primarily a hint for the programmer, as the compiler internally treats it as `int const *arr`.

# POINTER

## operations

# THE VALID POINTER OPERATIONS ARE

- Assignment of pointers to the same type,
- Adding or subtracting a pointer and an integer,
- Subtracting or comparing* two pointers,
- Assigning or comparing to zero (NULL = ‘\0’),
- All other pointer arithmetic is illegal (+, *, /, >>, <<, etc.).

# SUBTRACTING TWO POINTERS

The result of subtracting two pointers is the difference in their array indices, not the actual difference in their memory locations

```c
int strlen(char * string)
{
  char *pointer = string;
  while(*pointer != '\0') /* or NULL or 0 or FALSE */
    pointer++;
  return pointer - string; /* <===== */
}
```



```c
  char * text = "Hello world!";
  printf("Length of [%s] equals %d\n", text, strlen(text)); // Result: Length of [Hello world!] equals 12
```



# Comparing two pointers

Comparing pointers to strings in C can be significantly optimized, especially when there's a high probability that two strings are identical and point to the same memory location



```c
#include <stdio.h>
int main()
{
  char *str1 = "Hello";
  char *str2 = str1;
  if (str1 == str2)
        printf("Str1 and str2 point to the same string\n");
  else
  { // If addresses are different, then compare the content
    if ( strcmp(str1, str2) == 0 )
        printf("Str1 and str2 have the same content\n");
    else
        printf("Str1 and str2 are different\n");
  }
  return 0;
} // Result: Str1 and str2 point to the same string
```

# Comparing two pointers

- Array elements are pushed onto the stack in reverse order to ensure that element addresses increase.
- If we compare two elements of an array using two pointers, and we don't know which one is closer to the beginning and which one is closer to the end, the one with the higher address value will be closer to the end, and the one with the lower address value will be closer to the beginning of the array.

```c
#include <stdio.h>
int main()
{
  int x = 5, y = 7, z = 9;
  int b[]= { 1, 2, 3, 4 };
  char text[] = "hello";
  char *l1 = &text[2] , *l2 = &text[3];

  printf("%d.\n", &x);        // Result: 6422292.
  printf("%d.\n", &y);        // Result: 6422288.
  printf("%d.\n\n", &z);      // Result: 6422284.

  printf("%d.\n", &b[3]);     // Result: 6422280.
  printf("%d.\n", &b[2]);     // Result: 6422276.
  printf("%d.\n", &b[1]);     // Result: 6422272.
  printf("%d.\n\n", &b[0]);   // Result: 6422268.

  printf("%d.\n", &text[3]);  // Result: 6422265.
  printf("%d.\n", &text[2]);  // Result: 6422264.
  printf("%d.\n", &text[1]);  // Result: 6422263.
  printf("%d.\n\n", &text[0]); // Result: 6422262.

  if(l1 < l2)
    printf("l1 is closer to the beginning\n");
  else
    printf("l2 is closer to the beginning\n");
  return 0;
} // Result: l1 is closer to the beginning
```

# Assigning or comparing to zero (`NULL` == `\0`)

Trying to use a null pointer will crash the program.

```c
#include <stdio.h>
int main()
{
  char * a[10] = {NULL};

  a[0] = "Words";
  a[1] = "of";
  a[2] = "different";
  a[3] = "lengths";

  int i;
  for ( i = 0; i < 10; i++ )
    if(a[i])
      printf("%s ", a[i]);
  return 0;
} // Result: Words of different lengths
```



# THANK

You